

الجمهورية الجزائرية الديمقراطية الشعبية
وزارة التعليم العالي والبحث العلمي
المدرسة العليا للإعلام الآلي - ESISA
قسم هندسة البرمجيات

تقرير مشروع التخرج

نظام إدارة إيجار معدات المسرح الكبير

Grand Theatre Rental Management System

اسم الطالب	جيلالي حور - Djihlali Heur
رقم التسجيل	ESISA-2024-SW-0456
الفوج / السنة	السنة الثالثة - تخصص تطوير الويب
الأستاذ المشرف	د. محمد بن عيسى - Dr. M. Ben Aissa
المؤسسة	المسرح الكبير للجزائر - Grand Théâtre d'Alger
السنة الجامعية	2025 - 2026

سري - للاستخدام الأكاديمي فقط

1	مقدمة عامة وأهداف المشروع
2	بيانات الطالب والمؤسسة
3	المعمارية العامة للنظام
4	شرح تفصيلي لكل ملف ووحدة
4.1	الواجهة الخلفية - Django Backend
4.2	الواجهة الأمامية - Next.js Frontend
5	مخطط Merise - CDM
6	تدفق العمل والسيناريوهات
7	التقنيات المستخدمة
8	تعليمات التشغيل
9	الخلاصة والتوصيات

رقمنة وتحديث جميع العمليات المتعلقة بتأجير الأجهزة والمعدات، بدءاً من تلقي الطلبات وانتهاء بإصدار الفواتير وتوقيع العقود إلكترونياً. متكامل لإدارة إيجار المعدات المسرحية في المسرح الكبير للجزائر، وهو من أبرز الفضاءات الثقافية في المنطقة العربية. يهدف النظام إلى يعد هذا المشروع نظاماً

والفرنسية، ومراقبة الأنشطة عبر لوحة تحكم متقدمة. يأتي هذا النظام ليحل هذه الإشكاليات بأدوات تقنية حديثة ومنهجية هندسية رصينة. يواجه المسرح الكبير تحديات إدارية متعددة تشمل: تتبع المعدات والمخزون، إدارة العقود والعملاء، توليد الوثائق الرسمية بالعربية

الأهداف الرئيسية للمشروع:

- رقمنة إجراءات الإيجار من الطلب حتى الإرجاع
- إدارة المخزون ومتابعة حالة كل معدة في الوقت الفعلي
- توليد العقود وبطاقات الخروج والوصول بصيغة PDF احترافية
- نظام إشعارات فوري داخل التطبيق
- إدارة صلاحيات المستخدمين (المسؤول، المستأجر، الموظفون)
- دعم ثلاثة لغات: العربية، الفرنسية، الإنجليزية
- واجهة مستخدم متجاوبة مع الهاتف والحاسوب

معلومات الطالب

الاسم الكامل	Djihlali Heur / جيلالي حور
رقم التسجيل	ESISA-2024-SW-0456
السنة الدراسية	السنة الثالثة ليسانس - هندسة البرمجيات
الفوج	FG-07 / 2025-2026
البريد الإلكتروني	jihor4516@esisa.edu.dz
المشرف	د. محمد بن عيسى - Dr. Mohammed Ben Aissa
نوع المشروع	مشروع تطبيقي (تخرج)

معلومات المؤسسة المضيفة

اسم المؤسسة	المسرح الكبير للجزائر - Grand Théâtre d'Alger
النشاط	مرفق ثقافي وطني للفنون المسرحية والموسيقية
الموقع	ساحة الأمير عبد القادر - الجزائر العاصمة
الموقع الإلكتروني	www.grandtheatre.dz
مسؤول التدريب	أ. كمال بولحوارق
فترة الإنجاز	سبتمبر 2025 - مارس 2026

معلومات المؤسسة الأكاديمية

الاسم	المدرسة العليا للإعلام الآلي - ESISA
الوصاية	وزارة التعليم العالي والبحث العلمي
التخصص	هندسة البرمجيات - Génie Logiciel
العنوان	الجزائر العاصمة

المنفصلة (Decoupled Client-Server Architecture) مع واجهة برمجية RESTful JSON API. يتكون من طبقتين رئيسيتين: يعتمد النظام على معمارية العميل-الخادم

تعمل على المتصفح/الهاتف وتتواصل مع الخادم عبر HTTP/JSON	الواجهة الأمامية (Frontend)
والمنطق التجاري، API يوفر +40 نقطة نهاية — Django REST + Django 4.2 ويدير قاعدة البيانات	الواجهة الخلفية (Backend)
إنتاج) — تخزين جميع البيانات الدائمة) PostgreSQL / (تطوير) SQLite	قاعدة البيانات (Database)
متعددة اللغات PDF إلى ملفات HTML/CSS يحول قوالب — WeasyPrint احترافية	توليد PDF
بريد إلكتروني — لإرسال رموز التحقق والإشعارات + SMTP Django	الاتصالات

مبدأ التدفق (Request Flow):

المستخدم (متصفح/هاتف) → SQLite/PostgreSQL → ORM → Django Backend → REST JSON → Next.js Frontend → HTTPS

backend/manage.py

الهجرات، إنشاء مستخدمين، وتشغيل أوامر الإدارة. يقرأ إعدادات المشروع من متغير البيئة DJANGO_SETTINGS_MODULE. نقطة دخول Django. يستخدم لتشغيل الخادم، تطبيق

❏ لا يحتوي على منطق تجاري - الغرض منه تشغيل المشروع فقط.

backend/config/settings.py

مسارات الملفات الثابتة والوسائط، إعدادات البريد الإلكتروني SMTP، مفتاح الأمان SECRET_KEY، وإعدادات الترجمة (i18n). يضم: قائمة التطبيقات المثبتة (INSTALLED_APPS)، إعدادات قاعدة البيانات، إعدادات CORS للسماح لـ Next.js بالاتصال، قلب إعدادات المشروع.

❏ ينصح في الإنتاج بنقل SECRET_KEY وبيانات قاعدة البيانات إلى متغيرات البيئة (env).

backend/config/urls.py

، /api/ ❏ apps/api، /accounts/ ❏ apps/accounts، /contracts/ ❏ apps/contracts، /inventory/ ❏ apps/inventory، ملف توجيه URL الرئيسي. يربط عناوين URL بمسارات الوحدات الفرعية مثل: /documents/ ❏ apps/documents/

❏ يستخدم include () لتفويض التوجيه إلى كل تطبيق فرعي.

backend/config/asgi.py & wsgi.py

asgi.py يدعم الاتصالات غير المتزامنة (WebSocket/HTTP2)، بينما wsgi.py يستخدم للنشر التقليدي مع Gunicorn/uWSGI. ملفات نشر الخادم.

❏ يستخدم المشروع حالياً wsgi للتطوير و asgi للإنتاج.

backend/apps/accounts/models.py

LoginLog (سجل تسجيلات الدخول لأغراض الأمن)، VerificationCode (رموز التحقق من البريد الإلكتروني عند التسجيل)، (ملف العميل بأنواعه: فرد، شركة، جمعية، حكومة)، StaffProfile (الموظفون بأدوارهم: مدير، أمين مخزن، محاسب، مستشار)، UserProfile يعرف نماذج البيانات للمستخدمين: UserProfile

❏ يرتبط UserProfile و StaffProfile بـ Django User عبر علاقة OneToOne.

backend/apps/accounts/views.py

تسجيل الدخول بـ JWT/Session، تسجيل الخروج، عرض وتعديل الملف الشخصي، وإدارة المستخدمين من لوحة المشرف. يحتوي نماذج (views) تسجيل المستخدمين الجدد مع التحقق عبر البريد الإلكتروني،

يستخدم `send_verification_email` من `email_utils.py` لإرسال كود التحقق.

backend/apps/accounts/email_utils.py

البريد الإلكتروني. يحتوي على دوال لبناء وإرسال بريد التحقق باستخدام `django.core.mail.send_mail` مع قالب HTML مخصص. مساعد إرسال

يقرأ إعدادات SMTP من `settings.py`.

backend/apps/api/views.py

إدارة المعدات، العقود، الطلبات، المستأجرين، الإشعارات، والوثائق. يرجع البيانات بصيغة JSON مع دعم فلترة حسب اللغة. الملف الأضخم في المشروع (أكثر من 1100 سطر). يوفر جميع نقاط نهاية REST API: لوحة التحكم (dashboard)،

يستخدم `@require_GET/@require_POST` والصلاحيات المخصصة من `permissions.py`.

backend/apps/api/urls.py

يربط عناوين URL بدوال `views.py` الخاصة بـ API. تشمل: `/api/download-report/`، `/api/documents/`، `/api/notifications/`، `/api/tenants/`، `/api/requests/`، `/api/contracts/`، `/api/items/`، `/api/dashboard/`،

كل endpoint محمي بالتحقق من صلاحية المستخدم.

backend/apps/api/middleware.py

يعمل على كل طلب HTTP قبل وصوله للـ `view`. يسجل نشاط المستخدم ويحدث `last_activity` في `StaffProfile`. وسيط مخصص (Middleware)

يعمل تلقائياً بمجرد إدراجه في `MIDDLEWARE` بـ `settings.py`.

backend/apps/inventory/models.py

التسلسلي، الحالة، الكمية، السعر، موقع التخزين)، `ItemImage` (صور المعدة)، `MaintenanceLog` (سجل الصيانة والإصلاحات). يعرف `Category` (فئات المعدات مع دعم 4 لغات)، `Item` (المعدة بجميع خصائصها: الرقم

تدعم الكميات المتابعة التلقائية عند الإيجار والإرجاع.

backend/apps/contracts/models.py

وحالة الإرجاع)، ContractHistory (تاريخ تغييرات العقد للتدقيق)، RentalRequest (طلب الإيجار من العميل قبل إنشاء العقد). RentalContract (العقد الكامل بحالاته: مسودة، قيد الانتظار، نشط، منته)، ContractItem (بنود كل عقد مع الكميات والأسعار يعرف

□ يستخدم ManyToMany عبر جدول ContractItem لربط العقد بالمعدات.

backend/apps/contracts/pdf.py

الاسترداد (restitution) بالعربية والفرنسية. كما يحتوي على build_qr_data_uri () لإضافة رمز QR للتحقق من صحة الوثيقة. WeasyPrint لتحويل قوالب HTML إلى PDF. يوفر render_pdf () لتوليد عقد الإيجار ووثيقة

□ يحدد مسار ملف CSS في settings.BASE_DIR.

backend/apps/contracts/templates/pdf/contract.html

ترويسة المسرح، تفاصيل العاقدين، جدول المعدات المستأجرة، شروط العقد باللغتين، خانات التوقيع الإلكتروني، ورمز QR للتحقق. قالب HTML/CSS للعقد الرسمي المولد بـ PDF. يتضمن:

□ يستخدم CSS مضمن لضمان التوافق مع WeasyPrint.

backend/apps/documents/models.py

(بطاقة الاستقبال)، commande (أمر الشراء)، pv (محضر التسليم). كل وثيقة ترتبط بمعدة واحدة أو أكثر عبر DocumentItem. يعرف ExternalDocument بأنواعها الأربعة: decharge (وثيقة الخروج)، reception،

□ يتيح تتبع حركة المعدات بين المخزن والجهات الخارجية.

backend/apps/notifications/models.py

يحتوي على: sender (المرسل)، receivers (المستقبلون)، العنوان، النص، رابط الإجراء action_url، وحالة القراءة is_read. نموذج الإشعارات الداخلية.

□ تنشأ الإشعارات تلقائياً عند الأحداث المهمة (قبول طلب، انتهاء عقد...).

backend/apps/dashboard/models.py

نموذج SystemSettings لإعدادات النظام العامة كاسم الموقع، اللغة الافتراضية، وضع الصيانة، السماح بالتسجيل، ويريد التواصل.

□ يدار من لوحة الإدارة Django Admin.

backend/apps/accounts/permissions.py

و IsApprovedClient و CanManageContracts. كل صلاحية ترث Django Permission وتطبق على مستوى ال view. يعرف صلاحيات مخصصة ك IsStaffUser

□ يستخدم ك decorator أو في check_object_permissions.

frontend/app/layout.js

ترويسة الموقع (SiteHeader)، تذييله (SiteFooter)، استيراد styles عالمية globals.css، وإعدادات meta لل SEO. التخطيط الجذري لكل الصفحات (Root Layout). يحمل مرة واحدة ويحتوي على: مزود اللغة (LanguageProvider)،

□ يعمل ك Server Component لتحسين الأداء.

frontend/app/page.js

، FeaturedProducts، HeroBanner، و CtaPanel. مصممة لاستقطاب العملاء المحتملين وعرض خدمات المسرح. الصفحة الرئيسية للموقع العام. تركيب مكونات: CategoryGrid، ServiceHighlights، ProcessTimeline

□ تستخدم data/site.js لقراءة النصوص والصور.

frontend/app/catalogue/page.js

يجلب البيانات من /api/items/ ويعرضها في شبكة بطاقات مع فلاتر حسب الفئة، الحالة، والسعر. يدعم البحث النصي والترتيب. كتالوج المعدات المتاحة للإيجار.

□ يستخدم client-side fetching عبر lib/client-api.js.

frontend/app/items/[id]/page.js

صفحة تفاصيل معدة بعينها. تعرض: معرض الصور، الوصف متعدد اللغات، مواصفات المعدة، نموذج طلب الإيجار المدمج.

□ يستبدل برقم المعدة [id] ال - Next.js ديناميكي Route

frontend/app/tenant-dashboard/page.js

تحكم العميل المستأجر. تعرض: الطلبات الحالية وتاريخها، العقود النشطة، الإشعارات غير المقروءة، وإمكانية تحميل العقود PDF. لوحة

□ يتحقق من هوية المستخدم قبل العرض ويعيد التوجيه إلى /login إن لم يكن مسجلاً.

frontend/components/admin/staff-dashboard-page.js

مؤشرات KPI (طلبات، عقود، مستأجرون)، جدول الطلبات مع قبول/رفض، إدارة المعدات، العقود، وإرسال الإشعارات الجماعية. أضخم مكون في الواجهة (460 سطر). لوحة الإدارة الكاملة للموظفين تشمل:

□ يستخدم (setInterval Polling) لتحديث البيانات كل 30 ثانية.

frontend/components/auth/login-form.js

الدخول. يتعامل مع POST /accounts/login، يخزن التوكن في localStorage، ويوجه المستخدم حسب دوره (staff/client). نموذج تسجيل

□ يعالج أخطاء 401 و403 بإظهار رسائل مترجمة.

frontend/components/auth/signup-form.js

(245 سطر): بيانات الحساب، تحديد نوع المستخدم (فرد/شركة/جمعية)، رفع الوثائق المطلوبة، وإدخال كود التحقق البريدي. نموذج التسجيل متعدد الخطوات

□ يقسم الحقول ديناميكيا حسب نوع المستخدم المختار.

frontend/lib/client-api.js

...endpoint: fetchItems، fetchContracts، جميعها تضيف Authorization header تلقائيا وتعالج أخطاء الشبكة. طبقة التجريد للتواصل مع الـ API (145 سطر). تحتوي على دوال لكل createRentalRequest، updateContractStatus

□ يقرأ BACKEND_URL من متغير البيئة NEXT_PUBLIC_API_URL.

frontend/lib/backend-url.js

API بشكل ذكي: يستخدم المتغير البيئي في الإنتاج، ويعود لـ localhost:8000 في التطوير. يدعم SSR و Client-side rendering. يحدد عنوان

□ 73 سطرًا تضمن عمل الاتصال في جميع بيئات النشر.

frontend/lib/i18n.js & components/i18n/language-provider.js

i18next مع دعم العربية والفرنسية والإنجليزية. LanguageProvider يغلف التطبيق ويوفر useTranslation hook لكل المكونات. نظام الترجمة متعدد اللغات. i18n.js يهيئ

□ الترجمات مخزنة في frontend/locales/ar.json, fr.json, en.json.

frontend/app/globals.css

أنماط مكونات UI (buttons, cards, forms, tables)، تأثيرات hover/focus/active، media queries للأجهزة المحمولة. 948 سطرًا من CSS يغطي: نظام الألوان ومتغيرات CSS custom properties، تخطيط RTL/LTR،

يشكل الهوية البصرية الكاملة للموقع.

frontend/data/site.js

ملف البيانات الثابتة للموقع: أسماء الخدمات، النصوص التسويقية، قائمة الفئات، معلومات التواصل، وروابط التنقل.

يستخدم من صفحة الرئيسية وكتالوج الخدمات.

الأساسية بين كيانات النظام وفق منهجية Merise. تم تصميم هذا المخطط لإبراز الكيانات الرئيسية وخصائصها وطبيعة الروابط فيما بينها. يوضح النموذج المفاهيمي للبيانات (CDM) العلاقات

الكيانات (Entities):

User – المستخدم
id (PK)
username
password (hash)
email
first_name
last_name
is_staff
is_active
date_joined

UserProfile – ملف العميل
id (PK)
user_id (FK→User)
user_type
cin / company_name
phone
address
city
is_approved
is_blacklisted

StaffProfile – ملف الموظف
id (PK)
user_id (FK→User)
role
department
phone
is_active
theme_preference

language_preference

الفئة - Category

id (PK)

name_ar / name_fr / name_en

icon

color

slug

is_active

المعدة - Item

id (PK)

serial_number (UQ)

name_ar / name_fr / name_en

category_id (FK→Category)

status

condition

price_per_unit

price_period

quantity_total

quantity_available

storage_location

المستأجر - Tenant

id (PK)

user_id (FK→User)

tenant_type

organization_name

contact_person

phone

email

address

city

is_active

طلب الإيجار - RentalRequest

id (PK)

user_id (FK→User)

item_id (FKItem)
qty
event_name
start_date
end_date
status (pending/approved/rejected)
staff_note
reviewed_by (FKUser)

العقد - RentalContract

id (PK)
contract_number (UQ)
tenant_id (FKTenant)
start_date
end_date
event_name
event_type
venue
status
total_amount
payment_method
deposit_paid
created_by (FKUser)

بند العقد - ContractItem

id (PK)
contract_id (FKRentalContract)
item_id (FKItem)
quantity
unit_price
days
subtotal
returned
damage_charge

الوثيقة الخارجية - ExternalDocument

id (PK)
doc_number (UQ)

doc_type (decharge/reception/commande/pv)
external_name
purpose
status
created_by (FK User)

الإشعار - Notification

id (PK)
sender_id (FK User)
title
message
action_url
is_read
created_at

إعدادات النظام - SystemSettings

id (PK)
site_name
default_language
maintenance_mode
allow_registration
contact_email

العلاقات (Merise CDM) - Relations:

الكيان الأول	العلاقة / الأوزان	الكيان الثاني
User	يملك (0,1 : 1,1)	UserProfile
User	يملك (0,1 : 1,1)	StaffProfile
Item	ينتمي إلى (n,1 : 1,n)	Category
Tenant	يقدم (n : 1,1,1)	RentalRequest
RentalRequest	يرتبط بـ (n,1 : 1,1)	Item
Tenant	يبرم (n : 1,1,1)	RentalContract
RentalContract	يتضمن (n : 1,1,1)	ContractItem
ContractItem	يشير إلى (n,1 : 1,1)	Item
User	ينشئ (n : 1,1,1)	ExternalDocument
User	يرسل (n : 1,1,1)	Notification

أسطورة الرموز: PK = مفتاح أساسي ، FK = مفتاح خارجي ، UQ = فريد ، 1,1 = واحد لواحد ، n,1 = واحد لأكثر ، n,m = أكثر لأكثر

سيناريو 1: تسجيل عميل جديد

1. يزور العميل صفحة signup/ ويملاً نموذج التسجيل.
2. النظام يرسل كود 6 أرقام إلى بريده الإلكتروني.
3. العميل يدخل الكود في صفحة التحقق.
4. يتم إنشاء حساب + UserProfile بحالة is_approved=False.
5. مسؤول النظام يراجع الملف ويوافق عليه من لوحة التحكم.
6. العميل يتلقى إشعاراً بقبول طلبه ويمكنه الآن تقديم طلبات الإيجار.

سيناريو 2: إيجار معدة

1. العميل يتصفح الكتالوج ويختار معدة متاحة.
2. يملأ نموذج الطلب: التاريخ، المناسبة، الكمية.
3. ينشأ RentalRequest بحالة pending.
4. الموظف يتلقى إشعاراً ويراجع الطلب.
5. عند القبول: ينشأ RentalContract + ContractItem.
6. تولد وثيقة PDF للعقد مع QR code للتحقق.
7. تحدث quantity_available في Item تلقائياً.

سيناريو 3: إرجاع المعدة

1. في تاريخ الانتهاء: ينبه النظام الموظف.
2. الموظف يسجل إرجاع كل معدة (ContractItem.returned=True).
3. إن وجد ضرر: يسجل في damage_noted و damage_charge.
4. يولد محضر الاسترداد (PDF restitution).
5. تعاد الكمية لـ Item.quantity_available.
6. يغلق العقد بحالة completed.

الإصدار	الغرض	التقنية
4.2.29	إطار العمل الخلفى	Django
14 (React 18)	إطار العمل الأمامي	Next.js
68.1	توليد PDF	WeasyPrint
4.4.10	توليد PDF / الرسومات	ReportLab
1.0	خط عربي أكاديمي	Amiri Font
8.2	توليد QR Code	qrcode
3.0	تشكيل النص العربي	arabic_reshaper
الأخيرة	نظام الترجمة	i18next
أي إصدار	قاعدة بيانات التطوير	SQLite

تشغيل الواجهة الخلفية:

1. نسخ المستودع: git clone [رابط المستودع]
2. إنشاء بيئة افتراضية: python -m venv venv && venv\Scripts\activate
3. تثبيت المتطلبات: pip install -r requirements.txt
4. نسخ ملف الإعدادات: copy backend\.env.example backend\.env ثم عدل القيم
5. تطبيق الهجرات: python manage.py migrate
6. إنشاء مستخدم مسؤول: python manage.py createsuperuser
7. تشغيل الخادم: python manage.py runserver

تشغيل الواجهة الأمامية:

1. الانتقال للمجلد: cd frontend
 2. تثبيت الحزم: npm install
 3. نسخ متغيرات البيئة: copy .env.local.example .env.local
 4. تعديل NEXT_PUBLIC_API_URL=http://localhost:8000
 5. تشغيل الخادم: npm run dev
 6. فتح المتصفح على: http://localhost:3000
- ملاحظة: يمكن استخدام start-client.bat لتشغيل الخادمين معا على Windows.

البرمجيات تشمل: تصميم قاعدة بيانات علائقية، بناء REST API، تطوير واجهات تفاعلية، إدارة الصلاحيات، وتوليد الوثائق الرسمية آلياً. أسهم هذا المشروع في بناء تجربة عملية متكاملة في هندسة

أبرز الإنجازات:

- بناء نظام متكامل بـ +40 نقطة API من الصفر
- دعم 3 لغات كاملة (AR/FR/EN) في الواجهتين
- توليد وثائق PDF احترافية بالعربية والفرنسية
- نظام صلاحيات 5 مستويات محكم وآمن
- واجهة متجاوبة تعمل على الهاتف والحاسوب
- كود منظم وموثق قابل للصيانة والتوسعة

توصيات للتحسين المستقبلي:

- ترقية قاعدة البيانات إلى PostgreSQL في بيئة الإنتاج
- إضافة WebSocket للإشعارات الفورية بدلا من Polling
- بناء تطبيق موبايل Native بـ React Native
- إضافة تقارير إحصائية تفاعلية (Dashboard Analytics)
- تطبيق CI/CD مع Docker و GitHub Actions
- إضافة اختبارات تكاملية شاملة (Django TestCase + Jest)

نقدم هذا العمل ثمرة لجهود أكاديمي صادق، آمين أن يسهم في تحديث الإدارة الثقافية وتعزيز رقمنة المؤسسات الوطنية في الجزائر.